



RAPPORT DE PROJET VHDL

Préparé par :

Chayma Ben amor

Rimeh Ouledaleya

Classe :

2èmeGEC2

Enseignante :

Mme Salima Feki

TABLE DES MATIERES

Application 1 _____

Code

Simulation

Application 2 _____

Code

Simulation

Application 3 _____

Code

Simulation

Application 4 _____

Code

Simulation

APPLICATION1

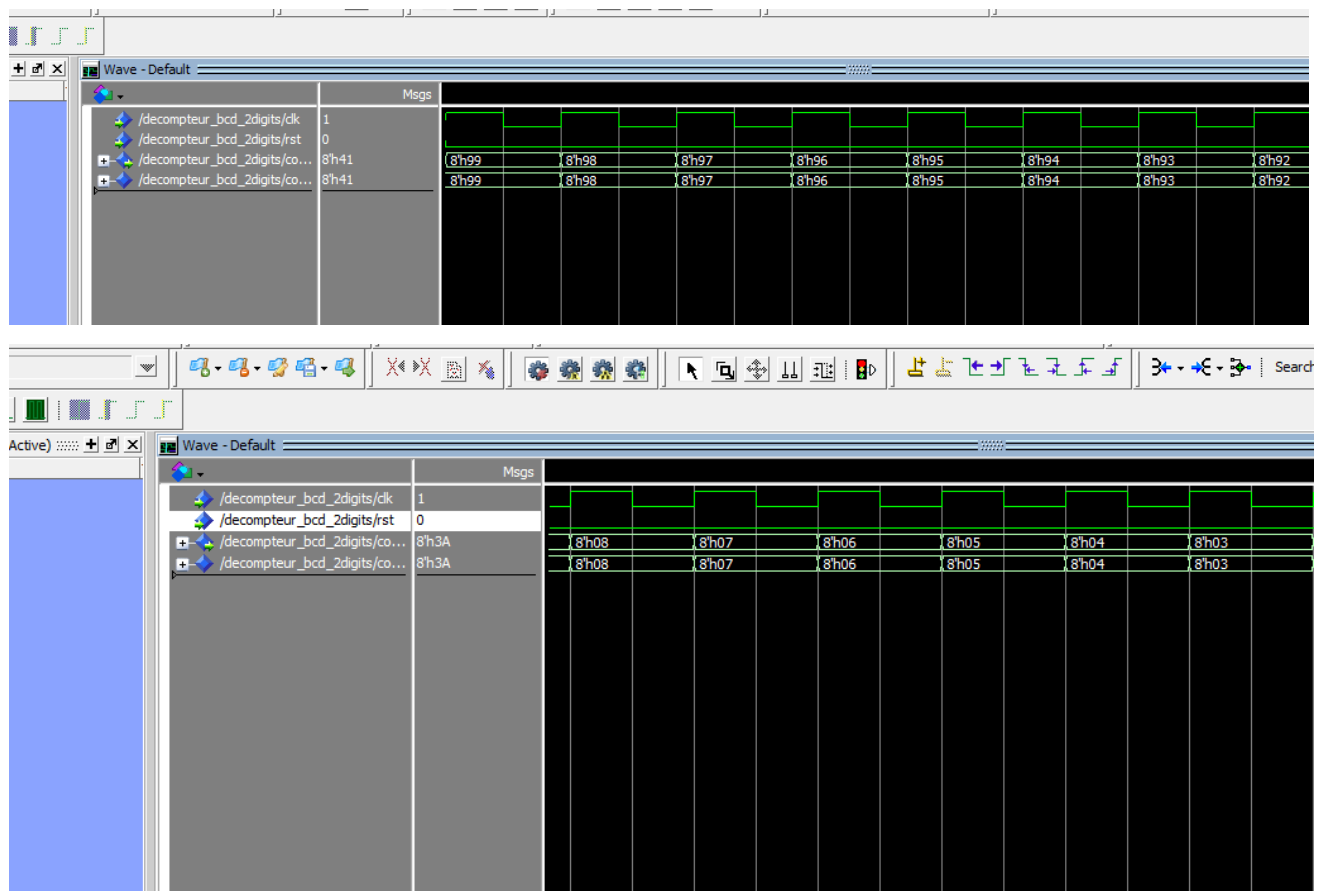
Code :

```

--/modeltech64_10.5/examples/app1.vhd (/decompteur_bcd_2digits) - Default
1#
2  use IEEE.STD_LOGIC_1164.ALL;
3  use IEEE.STD_LOGIC_ARITH.ALL;
4  use IEEE.STD_LOGIC_UNSIGNED.ALL;
5
6  entity Decompteur_BCD_2Digits is
7      Port (
8          clk : in  STD_LOGIC;
9          rst : in  STD_LOGIC;
10         count : out STD_LOGIC_VECTOR (7 downto 0)
11     );
12 end Decompteur_BCD_2Digits;
13
14 architecture Behavioral of Decompteur_BCD_2Digits is
15     signal count_reg : STD_LOGIC_VECTOR (7 downto 0) := "10011001"; -- 99 en BCD
16 begin
17     process (clk, rst)
18     begin
19         if rst = '1' then
20             count_reg <= "10011001"; -- Réinitialise le décompteur à 99
21         elsif rising_edge(clk) then
22             if count_reg = "00000000" then
23                 count_reg <= "10011001"; -- Retour à 99 après avoir atteint 0
24             else
25                 count_reg <= count_reg - 1; -- Décrémente le compteur
26             end if;
27         end if;
28     end process;
29
30     -- Assignment de la sortie
31     count <= count_reg;
32 end Behavioral;

```

Simulation :



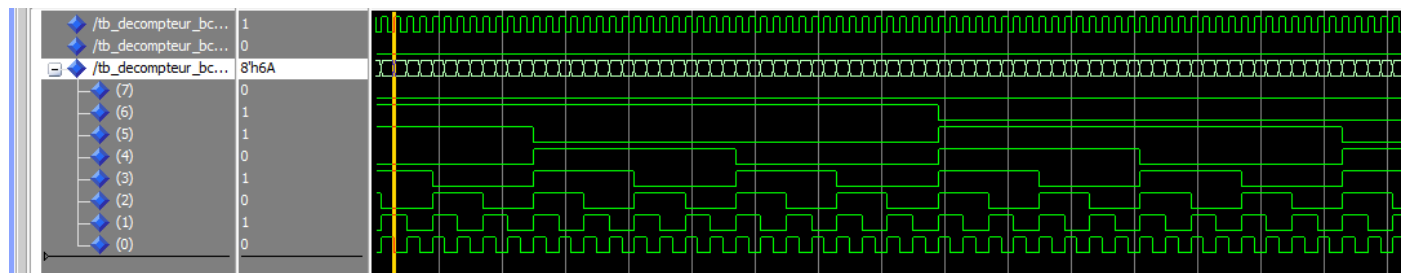
Test bench :

```

1  library IEEE;
2  use IEEE.STD_LOGIC_1164.ALL;
3  use IEEE.STD_LOGIC_UNSIGNED.ALL;
4  entity tb_Decompteur_BCD_2Digits_tb is
5  end tb_Decompteur_BCD_2Digits_tb;
6  architecture Behavioral of tb_Decompteur_BCD_2Digits_tb is
7      signal clk_tb    : STD_LOGIC := '0';
8      signal rst_tb    : STD_LOGIC := '0';
9      signal count_tb  : STD_LOGIC_VECTOR(7 downto 0);
10     constant clk_period : time := 10 ns;
11
12     begin
13         uut: entity work.Decompteur_BCD_2Digits
14             port map (
15                 clk    => clk_tb,
16                 rst    => rst_tb,
17                 count => count_tb );
18         clk_tb <= not clk_tb after clk_period / 2;
19         process
20             begin
21                 rst_tb <= '1';
22                 wait for clk_period;
23                 rst_tb <= '0';
24                 wait for 100 ns;
25
26                 rst_tb <= '1';
27                 wait for clk_period;
28                 rst_tb <= '0';
29
30                 wait for 100 ns;
31
32                 wait;
33             end process;
34     end Behavioral;

```

Simulation :



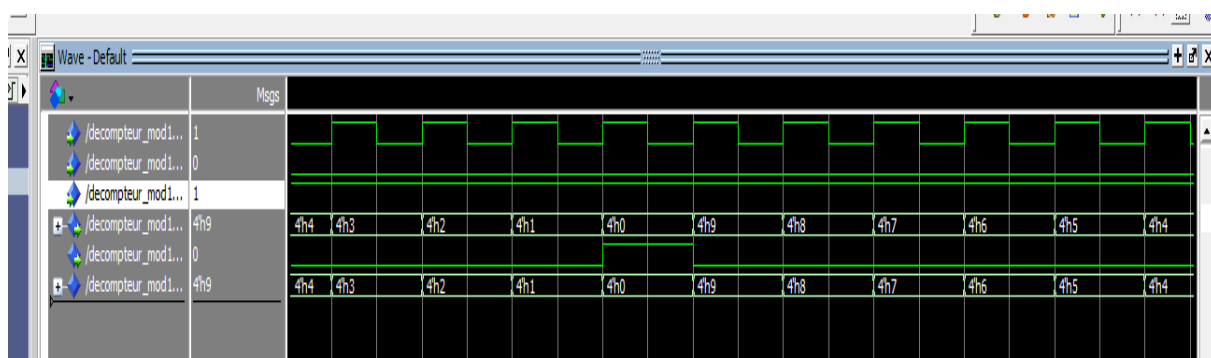
APPLICATION2

1)a)

```

C:/Users/Administrateur/Desktop/Nouveau dossier (2)/un.vhd (/tb_decompteur_mod10) - Default
Ln#
1  library IEEE;
2  use IEEE.STD_LOGIC_1164.ALL;
3  use IEEE.STD_LOGIC_ARITH.ALL;
4  use IEEE.STD_LOGIC_UNSIGNED.ALL;
5
6  entity decompteur_DCB_mod10 is
7      Port (
8          CLR : in STD_LOGIC;          -- Signal de remise à zéro
9          CLK : in STD_LOGIC;          -- Horloge
10         B0 : out STD_LOGIC;           -- Fin de cycle de décompte (0)
11         Q : out STD_LOGIC_VECTOR (3 downto 0) -- Valeur du compteur (Q)
12     );
13 end decompteur_DCB_mod10;
14
15 architecture Behavioral of decompteur_DCB_mod10 is
16     signal count : STD_LOGIC_VECTOR (3 downto 0) := "1001"; -- Initialisé à 9 (DCB modulo 10)
17 begin
18     process (CLK, CLR)
19     begin
20         if CLR = '1' then
21             count <= "1001"; -- Remise à 9
22         elsif rising_edge(CLK) then
23             if count = "0000" then
24                 count <= "1001"; -- Retour à 9 après décompte à 0
25             else
26                 count <= count - 1; -- Décrémentement du compteur
27             end if;
28         end if;
29     end process;
30
31     Q <= count;
32     B0 <= '1' when count = "0000" else '0'; -- Fin de cycle (B0) lorsqu'on atteint 0
33 end Behavioral;
34
35

```



b)

```

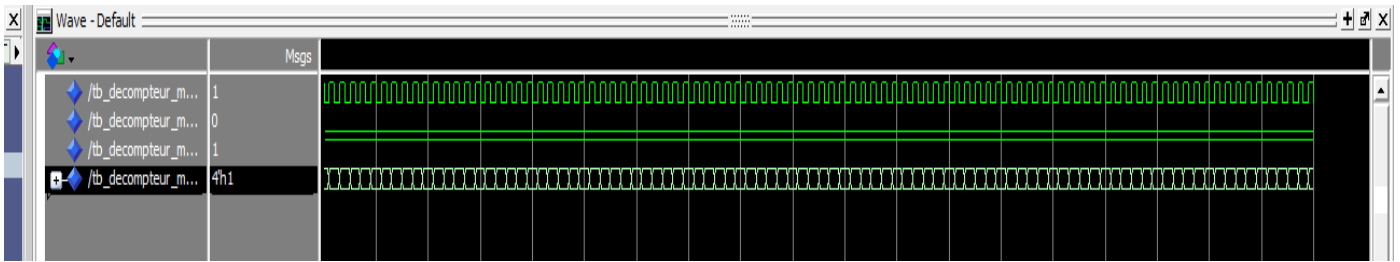
C:/Users/Administrateur/Desktop/Nouveau dossier (2)/tb_un.vhd (/tb_decompteur_mod10) - Default
Ln#
1  library IEEE;
2  use IEEE.STD_LOGIC_1164.ALL;
3
4  entity tb_decompteur_DCB_mod10 is
5  end tb_decompteur_DCB_mod10;
6
7  architecture behavior of tb_decompteur_DCB_mod10 is
8  -- Déclaration des signaux pour le test bench
9  signal CLR : STD_LOGIC := '0';
10 signal CLK : STD_LOGIC := '0';
11 signal B0 : STD_LOGIC;
12 signal Q : STD_LOGIC_VECTOR(3 downto 0);
13
14 -- Composant à tester
15 component decomp_teur_DCB_mod10
16 Port (
17     CLR : in STD_LOGIC;
18     CLK : in STD_LOGIC;
19     B0 : out STD_LOGIC;
20     Q : out STD_LOGIC_VECTOR (3 downto 0)
21 );
22 end component;
23
24 begin
25 -- Instanciation du décompteur
26 uut: decomp_teur_DCB_mod10
27 port map (
28     CLR => CLR,
29     CLK => CLK,
30     B0 => B0,
31     Q  => Q
32 );
33
34 -- Processus générant l'horloge
35 clk_process: process
36 begin
37     CLR <= '0';
38     wait for 10 ns;
39     CLR <= '1';
40     wait for 10 ns;
41 end process;
42
43 -- Stimuli
44 stim_proc: process
45 begin
46     -- Initialisation
47     CLR <= '0';
48     wait for 20 ns;
49
50     -- Test de remise à zéro
51     CLR <= '1';
52     wait for 20 ns;
53     CLR <= '0';
54     wait for 20 ns;
55
56     -- Test de décompte normal
57     wait for 100 ns; -- Attendre quelques cycles
58
59     -- Terminer la simulation
60     wait;
61 end process;
62 end behavior;
63
64
tb_un.vhd un.vhd deux.vhd *

```

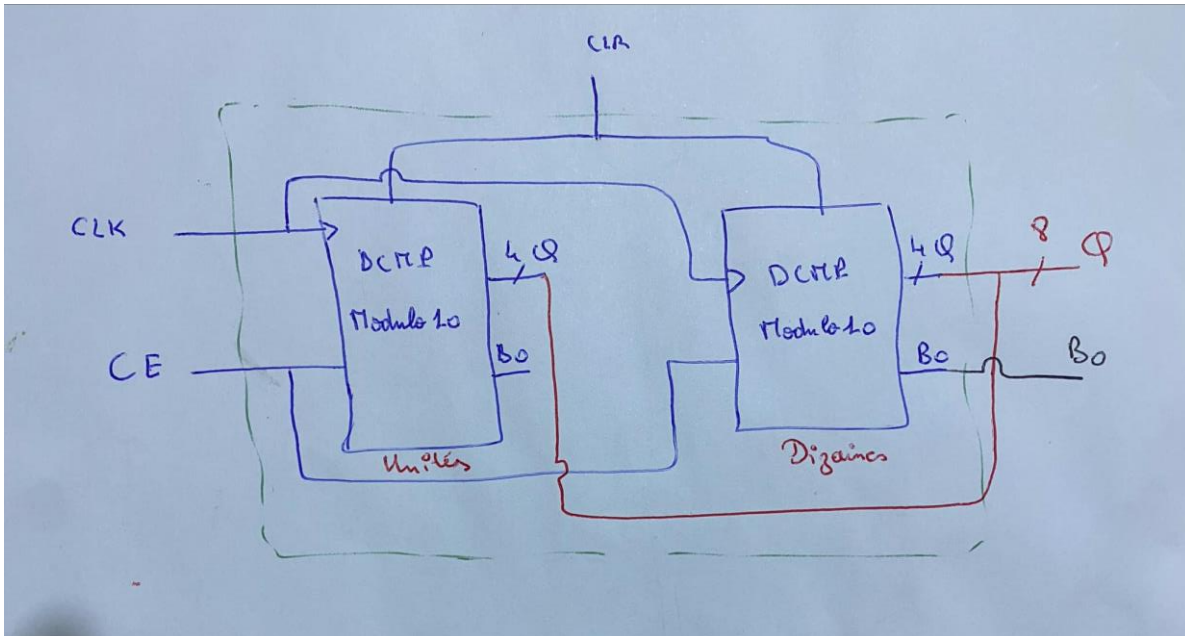
```

C:/Users/Administrateur/Desktop/Nouveau dossier (2)/tb_un.vhd (/tb_decompteur_mod10) - Def
Ln#
29     CLK => CLK,
30     B0 => B0,
31     Q  => Q
32 );
33
34 -- Processus générant l'horloge
35 clk_process: process
36 begin
37     CLR <= '0';
38     wait for 10 ns;
39     CLR <= '1';
40     wait for 10 ns;
41 end process;
42
43 -- Stimuli
44 stim_proc: process
45 begin
46     -- Initialisation
47     CLR <= '0';
48     wait for 20 ns;
49
50     -- Test de remise à zéro
51     CLR <= '1';
52     wait for 20 ns;
53     CLR <= '0';
54     wait for 20 ns;
55
56     -- Test de décompte normal
57     wait for 100 ns; -- Attendre quelques cycles
58
59     -- Terminer la simulation
60     wait;
61 end process;
62 end behavior;
63
64
tb_un.vhd un.vhd deux.vhd *

```



2)a)



b)

```

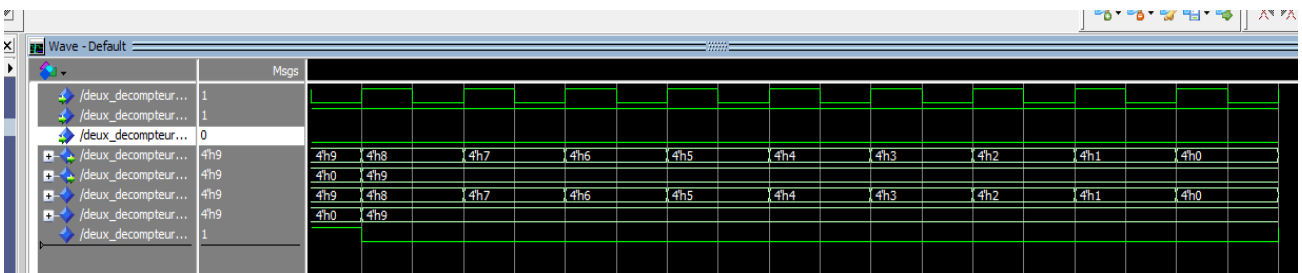
C:/DCB/question.vhd - Default *
1  library IEEE;
2  use IEEE.STD_LOGIC_1164.ALL;
3  use IEEE.STD_LOGIC_ARITH.ALL;
4  use IEEE.STD_LOGIC_UNSIGNED.ALL;
5  entity decompteur_deux_digits is
6
7      Port (
8
9          CLK : in STD_LOGIC;
10
11         CE : in STD_LOGIC;
12
13         CLR : in STD_LOGIC;
14
15         Q : out STD_LOGIC_VECTOR(7 downto 0);
16         B0 : out STD_LOGIC);
17 end decompteur_deux_digits;
18
19 architecture Structural of decompteur_deux_digits is
20
21     signal units_Q : STD_LOGIC_VECTOR(3 downto 0);
22
23     signal tens_Q : STD_LOGIC_VECTOR(3 downto 0);
24     signal B0_1, B0_2 : STD_LOGIC;
25
26     signal CE_tens : STD_LOGIC := '0';
27
28
29     component decompteur_DCB
30
31     Port (
32
33         CLK : in STD_LOGIC;
34
35         CE : in STD_LOGIC;
36
37         CLR : in STD_LOGIC;
38
39         B0 : out STD_LOGIC;
40
41         Q : out STD_LOGIC_VECTOR(3 downto 0)
42
43     );

```

```

43      );
44      end component;
45      begin
46          U1 : decompteur_DCB
47              port map (
48                  CLK => CLK,
49                  CE  => CE,
50                  CLR => CLR,
51                  B0  => B0_1,
52                  Q   => units_Q );
53          U2 : decompteur_DCB
54              port map (
55                  CLK => CLK,
56                  CE  => CE_tens,
57                  CLR => CLR,
58                  B0  => B0_2,
59                  Q   => tens_Q
60              );
61
62          process (CLK)
63          begin
64              if rising_edge(CLK) then
65                  if B0_1 = '1' then
66                      CE_tens <= '1';
67                  end if;
68              end if;
69          end process;
70          Q <= tens_Q & units_Q;
71          B0 <= B0_2;
72      end Structural;

```



c)

```

Ln#      1  library IEEE;
          2  use IEEE.STD_LOGIC_1164.ALL;
          3  use IEEE.STD_LOGIC_ARITH.ALL;
          4  use IEEE.STD_LOGIC_UNSIGNED.ALL;
          5  entity deux_decompteur_2_digits_tb is
          6  end deux_decompteur_2_digits_tb;
          7  architecture Behavioral of deux_decompteur_2_digits_tb is
          8      signal CLK      : STD_LOGIC := '0';
          9      signal CE       : STD_LOGIC := '1';
         10      signal CLR      : STD_LOGIC := '0';
         11      signal Bo_units : STD_LOGIC_VECTOR(3 downto 0);
         12      signal Bo_tens : STD_LOGIC_VECTOR(3 downto 0);
         13      signal B0_units : STD_LOGIC;
         14      signal B0_tens  : STD_LOGIC;
         15
         16      component deux_decompteur_2_digits
         17
         18          Port (
         19
         20              CLK      : in STD_LOGIC;
         21
         22              CE       : in STD_LOGIC;
         23
         24              CLR      : in STD_LOGIC;
         25
         26              Bo_units : out STD_LOGIC_VECTOR (3 downto 0);
         27
         28              Bo_tens  : out STD_LOGIC_VECTOR (3 downto 0);
         29
         30          );
         31
         32      end component;
         33
         34      begin
         35
         36          UUT: deux_decompteur_2_digits
         37
         38          port map (
         39
         40              CLK => CLK,
         41
         42              CE  => CE,

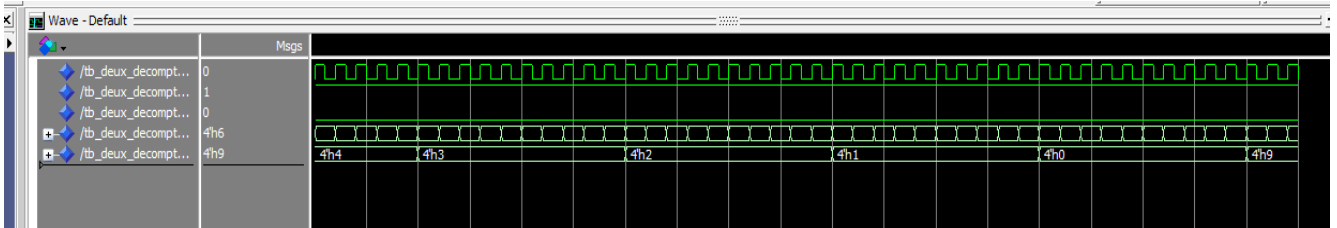
```



```

44         CLR => CLR,
45         Bo_units => Bo_units,
46         Bo_tens => Bo_tens );
47     CLK_process : process
48     begin
49         CLK <= '0';
50         wait for 10 ns;
51         CLK <= '1';
52         wait for 10 ns;
53     end process;
54     stim_proc: process
55     begin
56         CLR <= '1';
57
58         wait for 20 ns;
59
60         CLR <= '0';
61
62         CE <= '1';
63
64         wait for 100 ns;
65
66         wait for 200 ns;
67
68         CE <= '0';
69
70         wait for 50 ns;
71
72         CE <= '1';
73
74         wait for 100 ns;
75
76         wait;
77     end process;
78
79
80
81
82 end Behavioral;
83

```



APPLICATION3

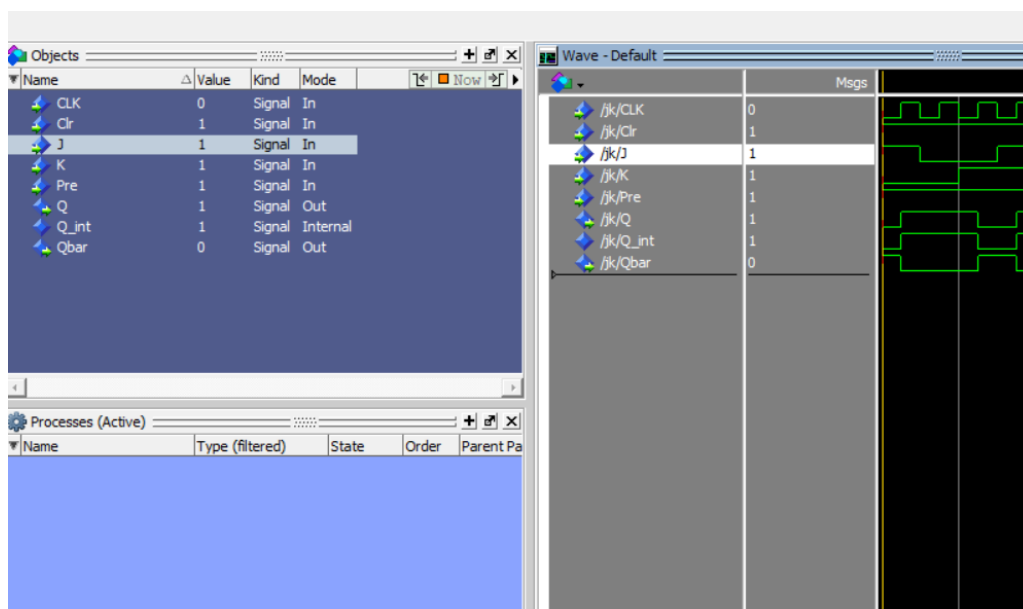
Code :

Bascula jk

```

C:/compdecompapp3/jk.vhd (jk) - Default
Ln#
1  library IEEE;
2  use IEEE.STD_LOGIC_1164.ALL;
3
4  entity JK is
5  port(
6      J, K, CLK, Pre, Clr : in std_logic;
7      Q, Qbar : out std_logic );
8  end entity;
9
10 architecture Behavioral of JK is
11     signal Q_int : std_logic := '0';
12 begin
13     process (CLK, Pre, Clr)
14     begin
15         if Pre = '0' then
16             Q_int <= '1';
17         elsif Clr = '0' then
18             Q_int <= '0';
19         elsif rising_edge(CLK) then
20             if J = '1' and K = '0' then
21                 Q_int <= '1';
22             elsif J = '0' and K = '1' then
23                 Q_int <= '0';
24             elsif J = '1' and K = '1' then
25                 Q_int <= not Q_int;
26             end if;
27         end if;
28     end process;
29     Q <= Q_int;
30     Qbar <= not Q_int;
31 end Behavioral;
32
33

```



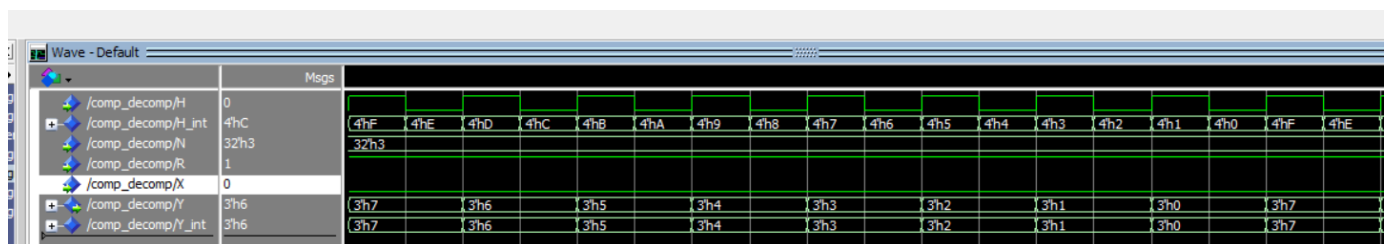
Compteur / décompteur

```

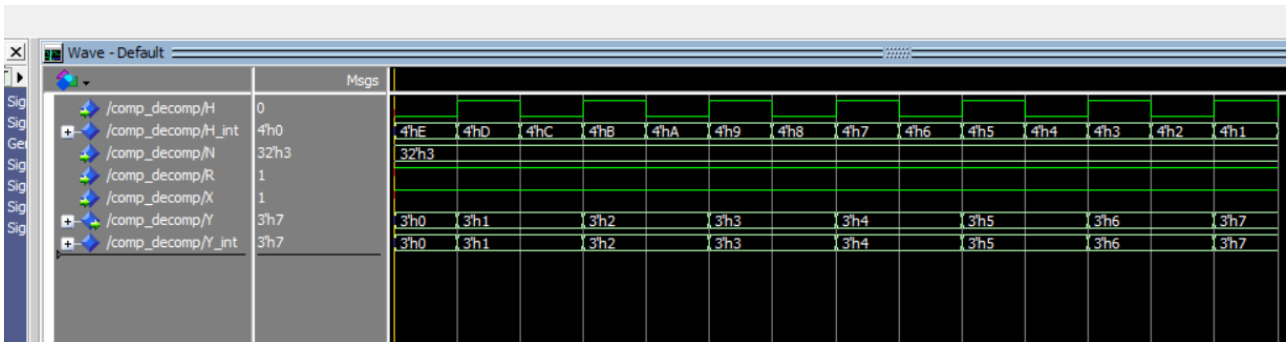
C:/compdecompapp3/compdecomp2.vhd (/comp_decomp) - Default
Ln#
1  library IEEE;
2  use IEEE.STD_LOGIC_1164.ALL;
3
4  entity Comp_Decompile is
5      generic(N : integer := 3);
6      port(
7          H : in STD_LOGIC;
8          R : in STD_LOGIC;
9          X : in STD_LOGIC;
10         Y : out STD_LOGIC_VECTOR(N-1 downto 0)
11     );
12 end entity;
13
14 architecture mixte of Comp_Decompile is
15     component JK is
16     port(
17         J, K, CLK, Pre, Clr : in STD_LOGIC;
18         Q, Qbar : out STD_LOGIC
19     );
20 end component;
21
22     signal H_int : STD_LOGIC_VECTOR(N downto 0);
23     signal Y_int : STD_LOGIC_VECTOR(N-1 downto 0);
24 begin
25     boucle: for i in 0 to N-1 generate
26     ul: JK port map(
27         J => '1',
28         K => '1',
29         CLK => H_int(i),
30         Pre => '1',
31         Clr => R,
32         Q => Y_int(i),
33         Qbar => open
34     );
35     H_int(i+1) <= Y_int(i) xor X;
36 end generate;
37
38     H_int(0) <= H;
39     Y <= Y_int;
40 end architecture;
41

```

Décompteur X=0



Compteur X=1



Test Bench

```

C:/compdecompapp3/testbench.vhd (/tb_comp_decomp) - Default
Ln#
1  library IEEE;
2  use IEEE.STD_LOGIC_1164.ALL;
3  entity TB_Comp_Decomp is
4  end entity;
5  architecture Behavioral of TB_Comp_Decomp is
6  component Comp_Decomp is
7      generic(N : integer := 3);
8      port(
9          H : in STD_LOGIC;
10         R : in STD_LOGIC;
11         X : in STD_LOGIC;
12         Y : out STD_LOGIC_VECTOR(N-1 downto 0)
13     );
14 end component;
15 signal H : STD_LOGIC := '0';
16 signal R : STD_LOGIC := '1';
17 signal X : STD_LOGIC := '1';
18 signal Y : STD_LOGIC_VECTOR(2 downto 0);
19 constant clk_period : time := 50 ns;
20 begin
21     UUT: Comp_Decomp
22         generic map(N => 3)
23         port map(
24             H => H,
25             R => R,
26             X => X,
27             Y => Y );
28     clk_process: process
29     begin
30         while true loop
31             H <= '0';
32             wait for clk_period / 2;
33             H <= '1';
34             wait for clk_period / 2;
35         end loop;
36     end process;
37     stim_process: process
38     begin
39         wait for 20 ns;
40         R <= '0';
41         wait for clk_period;
42         R <= '1';
43         X <= '1';

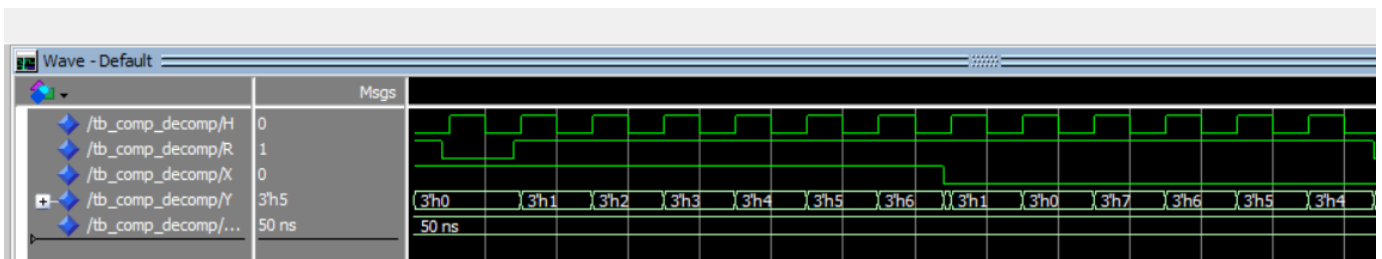
```

```

43     X <= '1';
44     wait for 300 ns;
45     X <= '0';
46     wait for 300 ns;
47     R <= '0';
48     wait for clk_period;
49     R <= '1';
50     wait for 200 ns;
51     assert false report "Fin de la simulation." severity note;
52     wait;
53 end process;
54 end architecture;
55
56

```

Simulation :



APPLICATION 4

1) Diagramme à états finis

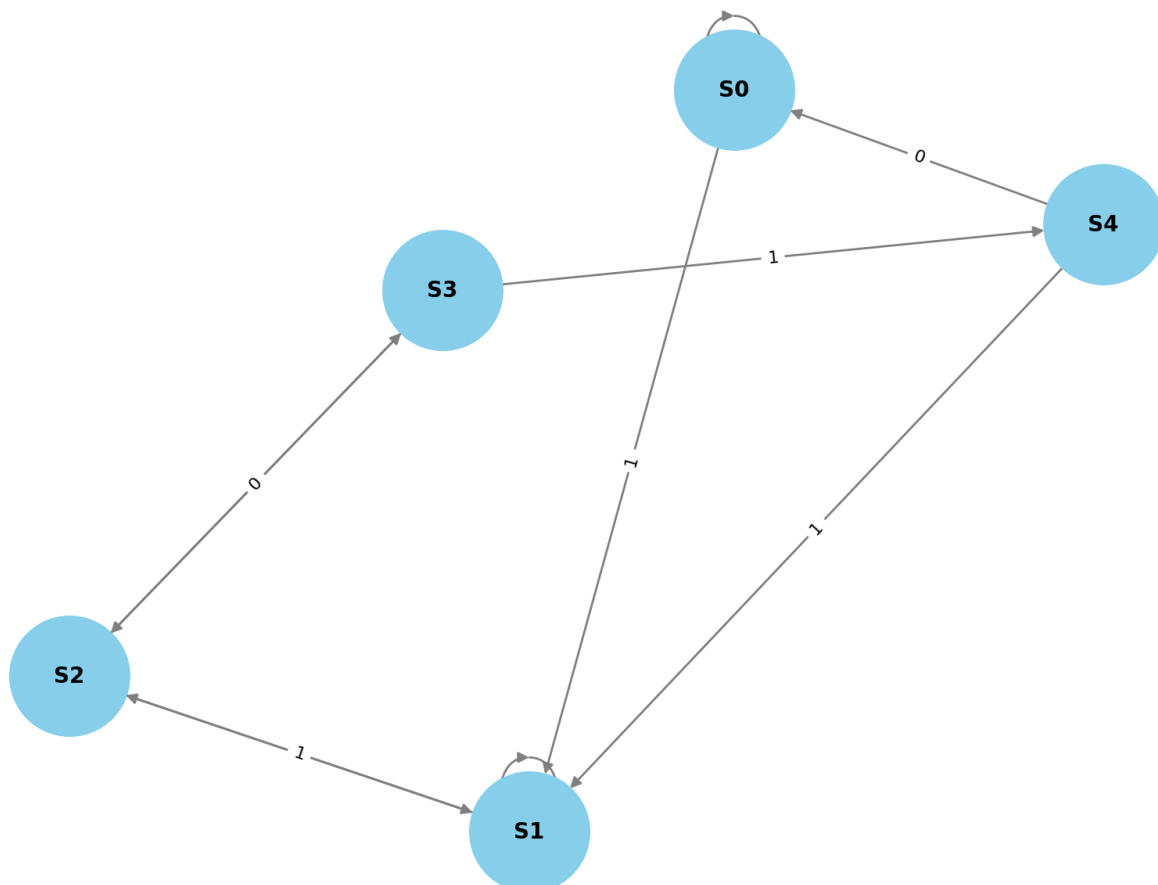
Le détecteur de séquence "1011" est conçu en utilisant une machine à états finis (FSM). Voici les états :

- **État 0 (S0)** : État initial, en attente d'un 1.
- **État 1 (S1)** : Détecte le premier 1.
- **État 2 (S2)** : Détecte 10.
- **État 3 (S3)** : Détecte 101.
- **État 4 (S4)** : Détecte 1011 (séquence complète).

Transition entre états :

- $S0 \rightarrow S1$: Si entrée = 1.
- $S1 \rightarrow S2$: Si entrée = 0.
- $S2 \rightarrow S3$: Si entrée = 1.
- $S3 \rightarrow S4$: Si entrée = 1.
- $S4 \rightarrow S1$: Pour détecter les chevauchements.

Diagramme d'états finis pour le détecteur de séquence '1011'



Code :

2)

```

1  library IEEE;
2  use IEEE.STD_LOGIC_1164.ALL;
3  use IEEE.STD_LOGIC_ARITH.ALL;
4  use IEEE.STD_LOGIC_UNSIGNED.ALL;
5
6  entity Sequence_Detector is
7  Port (
8      clk      : in  STD_LOGIC;
9      reset    : in  STD_LOGIC;
10     din      : in  STD_LOGIC;
11     dout     : out STD_LOGIC
12 );
13 end Sequence_Detector;
14
15 architecture Behavioral of Sequence_Detector is
16     type state_type is (S0, S1, S2, S3, S4);
17     signal current_state, next_state: state_type;
18
19 begin
20     process(clk, reset)
21     begin
22         if reset = '1' then
23             current_state <= S0;
24         elsif rising_edge(clk) then
25             current_state <= next_state;
26         end if;
27     end process;
28     process(current_state, din)
29     begin
30         case current_state is
31             when S0 =>
32                 if din = '1' then
33                     next_state <= S1;
34                 else
35                     next_state <= S0;
36                 end if;
37             when S1 =>
38                 if din = '0' then
39                     next_state <= S2;
40                 else
41                     next_state <= S1;
42                 end if;
43

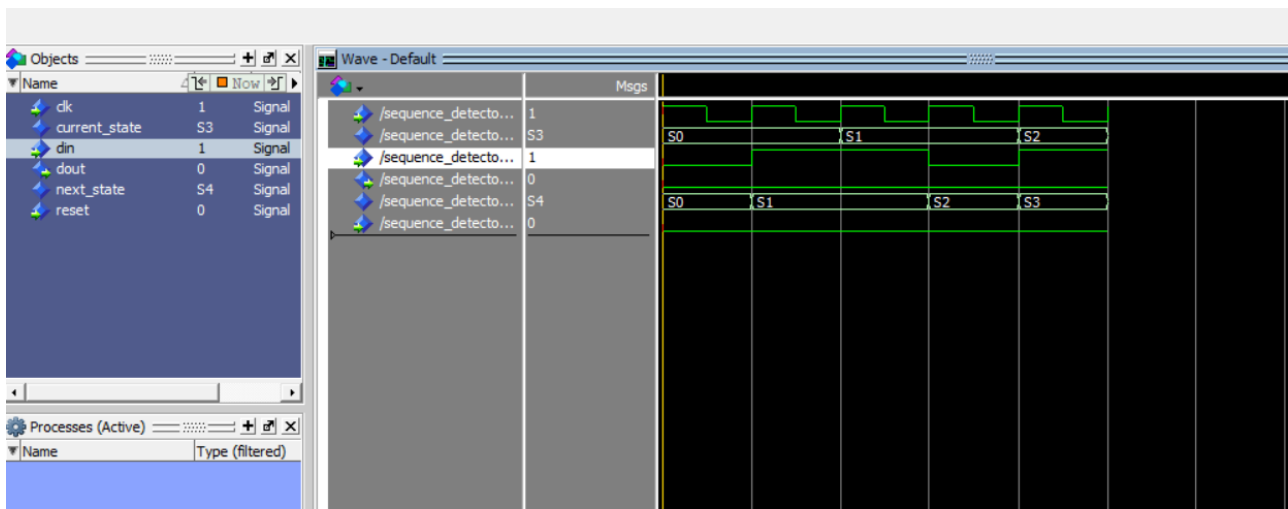
```

```

43         end if;
44     when S2 =>
45         if din = '1' then
46             next_state <= S3;
47         else
48             next_state <= S0;
49         end if;
50
51     when S3 =>
52         if din = '1' then
53             next_state <= S4;
54         else
55             next_state <= S2;
56         end if;
57
58     when S4 =>
59         if din = '1' then
60             next_state <= S1;
61         else
62             next_state <= S0;
63         end if;
64
65     when others =>
66         next_state <= S0;
67     end case;
68 end process;
69 dout <= '1' when current_state = S4 else '0';
70
71 end Behavioral;
72

```

Simulation :



Test bench :

3)

```

C:/vhdl projet/work/dtest.vhd (/tb_sequence_detector) - Default
[Icons]

Ln# |
1  | library IEEE;
2  | use IEEE.STD_LOGIC_1164.ALL;
3  | use IEEE.STD_LOGIC_ARITH.ALL;
4  | use IEEE.STD_LOGIC_UNSIGNED.ALL;
5  |
6  | entity tb_Sequence_Detector is
7  | end tb_Sequence_Detector;
8  |
9  | architecture Behavioral of tb_Sequence_Detector is
10 |
11 |
12 |     component Sequence_Detector
13 |     Port (
14 |         clk      : in  STD_LOGIC;
15 |         reset    : in  STD_LOGIC;
16 |         din      : in  STD_LOGIC;
17 |         dout     : out STD_LOGIC
18 |     );
19 | end component;
20 |
21 |
22 |     signal clk      : STD_LOGIC := '0';
23 |     signal reset    : STD_LOGIC := '0';
24 |     signal din      : STD_LOGIC := '0';
25 |     signal dout     : STD_LOGIC;
26 |
27 |
28 |     constant clk_period : time := 50 ns;
29 |
30 | begin
31 |
32 |
33 |     uut: Sequence_Detector
34 |     port map (
35 |         clk      => clk,
36 |         reset    => reset,
37 |         din      => din,
38 |         dout     => dout
39 |     );
40 |
41 |
42 |     clk_process : process
43 |     begin

```

```

43     begin
44         clk <= '0';
45         wait for clk_period / 2;
46         clk <= '1';
47         wait for clk_period / 2;
48     end process;
49     stimulus_process : process
50     begin
51         reset <= '1';
52         wait for clk_period;
53         reset <= '0';
54         din <= '0'; wait for clk_period;
55         din <= '1'; wait for clk_period;
56         din <= '0'; wait for clk_period;
57         din <= '1'; wait for clk_period;
58         din <= '0'; wait for clk_period;
59         din <= '1'; wait for clk_period;
60         din <= '1'; wait for clk_period;
61         din <= '1'; wait for clk_period;
62         din <= '0'; wait for clk_period;
63         din <= '1'; wait for clk_period;
64         din <= '1'; wait for clk_period;
65         din <= '0'; wait for clk_period;
66         din <= '1'; wait for clk_period;
67         din <= '1'; wait for clk_period;
68         wait;
69     end process;
70
71 end Behavioral;

```

Simulation :

